# C++ Programming Problem Set

Module: Structures, Encapsulation & User-Defined Types

CS-SIMPLIFIED

**Author:** Gaurav Kandpal

**Target Concept:** Structures & OOP Foundations

**Integrates:** I/O, Functions, Pointers, References, Arrays & Strings

**Instructions & Pedagogical Guidelines**

This problem set focuses on C++ user-defined structures. You will solve problems comparing procedural C-style external functions against modern C++ encapsulated member methods, memory layouts with padding, arrays of structures, pass-by-reference/pointer member access (`->` operator), nested structures, and dynamic heap allocation of struct objects.

## Level 1: Basic Declarations, Member Functions & Memory Layout

### Problem 1: Procedural C-Style vs Encapsulated C++ Struct

C vs C++ Structs | Member Functions | Encapsulation

Define a structure `Book` containing `bookID` (int), `title` (char[30]), and `price` (float). Implement two versions:
1. Procedural C-style function: `Book inputBook()` and `void displayBook(Book b)` taking/returning structs as parameters.
2. Encapsulated C++ style: Internal member functions `void input()` and `void display()` that access fields directly.
Demonstrate in `main()` how C++ member functions eliminate passing global struct copies.

Encapsulation | Direct Variable Access | Dot (.) Operator

| Implementation Approach | Execution Behavior |
|---|---|
| `Procedural (C-Style)` | `Requires external function: displayBook(b1) (Passes complete struct copy)` |
| `Encapsulated (C++)` | `Invokes member method: b1.display() (Direct internal memory access)` |

### Problem 2: Structure Byte Alignment & RAM Address Analyzer

Memory Layout | sizeof() | Byte Padding

Define a structure `DataHolder` containing `char c1`, `int iVal`, `char c2`, and `double dVal`. Create an instance in `main()`. Output the address of each member using pointer casting, and compare the sum of individual member byte sizes against `sizeof(DataHolder)`. Explain how compiler byte alignment padding creates contiguous RAM offsets without altering logical member order.

sizeof() Operator | Memory Padding | Contiguous Offsets

| Member Breakdown | Observed Memory Offset |
|---|---|
| `char c1 (1 Byte), int iVal (4 Bytes)` | `Sum of bytes = 14 \| sizeof(DataHolder) = 24 Bytes (Padding applied!)` |

### Problem 3: Pass-by-Reference Struct Mutator & Arrow Operator

Pass-by-Reference | Pointers | Arrow (->) Operator

Define a `Rectangle` struct with `length` and `width`. Write two external helper functions:
1. `void scaleByRef(Rectangle &r, double factor)` using reference variables.
2. `void scaleByPtr(Rectangle *r, double factor)` using pointer dereferencing via the arrow operator (`r->length`).
Demonstrate in `main()` that both functions mutate the original struct instance in RAM without copy overhead.

Arrow (->) Operator | Pass-by-Reference Alias | Pointer Access

| Initial Struct | Mutated Output |
|---|---|
| `Rect: len = 10, wid = 5` | `After scaleByPtr(&r, 2.0): len = 20, wid = 10 (Direct RAM modification)` |

## Level 2: Arrays of Structs, Sorting & Nested Structures

### Problem 4: Array of Structs Student Grade Manager

Array of Structs | Sorting | Member Search

Create a structure `Student` with `rollNo` (int), `name` (std::string), and `marks` (double). Write a program that inputs an array of 5 students. Implement functions:
1. `void sortStudentsByMarks(Student arr[], int size)` to sort the array descending.
2. `int findTopScorer(const Student arr[], int size)` returning the index of the highest scorer.
Print a formatted merit list report from `main()`.

```
// Array of Structs Management: struct Student { int rollNo; std::string name; double marks; }; void sortStudentsByMarks(Student arr[], int size);
```

| Input Records | Sorted Output Report |
|---|---|
| `[Roll 1: Alice (85), Roll 2: Bob (92)]` | `Rank 1: Bob (92.0) \| Rank 2: Alice (85.0)` |

### Problem 5: Nested Structure Hierarchy (Employee & Address)

Nested Structs | Deep Member Access | Strings

Design a nested structure layout. Define a structure `Address` with `city` (std::string) and `zipCode` (int). Define a outer structure `Employee` containing `empID` (int), `name` (std::string), and an embedded `Address homeAddr`. Write member functions inside `Employee` to read and display full details, accessing nested fields via dot chaining (`emp.homeAddr.city`).

Nested Struct Blueprint | Dot Chaining Access | Composition Pattern

| Sample Employee Input | Nested Output Display |
|---|---|
| `ID: 101, Name: "Gaurav", City: "Agra", Zip: 282001` | `Emp #101: Gaurav \| Address: Agra (ZIP: 282001)` |

### Problem 6: Time Difference Calculator with Struct Member Overloads

Struct Member Functions | Arithmetic Processing | Validation

Define a structure `Time` with `hours`, `minutes`, and `seconds`. Add member functions:
1. `void normalize()` — Adjusts excess seconds/minutes into higher units (e.g., 70s -> 1m 10s).
2. `Time subtract(const Time &t2)` — Returns a new `Time` struct representing the absolute duration difference.
Prompt the user for two time durations in `main()`, normalize inputs, and display the calculated time difference.

Struct Self-Normalization | Returning Struct Objects | Duration Math

| Input Durations | Normalized Difference |
| --- | --- |
| T1: 02:15:30 \| T2: 01:45:50 | Difference: 00:29:40 (Normalized accurately) |

## Level 3: Dynamic Memory, Constructors & Capstone Project

### Problem 7: Dynamic Struct Pointer Array Allocation (new / delete)

Dynamic Memory | new/delete | Struct Pointers

Create a program that dynamically allocates an array of structures on the Heap using new. Prompt the user for the number of items N. Dynamically allocate `Product *catalog = new Product[N];`. Read product details (`id`, `name`, `price`) using arrow operator indexing `(catalog + i)->input()`. Compute total catalog inventory value, display results, and free allocated Heap memory using `delete[] catalog;`.

```
// Dynamic Heap Allocation of Struct Array: Product *catalog = new Product[N]; for(int i = 0; i < N; i++) { (catalog + i)->input(); } delete[] catalog; // Essential memory cleanup!
```

| Heap State | Inventory Valuation Summary |
| --- | --- |
| Allocated N=3 Product items on Heap | Total Catalog Value: $1,450.00 \| Heap memory deallocated cleanly. |

### Problem 8: Struct Constructor Initialization & Default Arguments

Struct Constructors | Default Arguments | Initialization

In C++, structures support constructor functions! Define a structure `ComplexNumber` with `real` and `imag` fields. Add a parameterized constructor with default arguments: `ComplexNumber(double r = 0.0, double i = 0.0)`. Implement member functions `ComplexNumber add(const ComplexNumber &other)` and `void display()`. Instantiate instances with 0, 1, and 2 arguments to test automatic initialization.

Struct Constructors | Default Parameter Inits | Complex Arithmetic

| Call Site | Initialized State Output |
| --- | --- |
| ComplexNumber c1(5.0); | Value: 5 + 0i (Default imag initialized) |
| c1.add(ComplexNumber(2.0, 3.0)) | Result: 7 + 3i |

### Problem 9: Cumulative Capstone: Complete Banking Ledger Engine

Comprehensive Integration | Nested Structs | Batch Engine

Build a complete banking transaction ledger system. Define a nested structure hierarchy:
• `Transaction`: `txID` (int), `amount` (double), `type` (char: 'D'/'W').
• `Account`: `accNo` (int), `holderName` (std::string), `balance` (double), `txHistory[10]` (array of nested `Transaction` structs), and `txCount` (int).
Implement member functions: `void deposit(double amt)`, `void withdraw(double amt)`, and `void printStatement()`. Process batch user transactions in `main()` and display a complete account audit statement.

Nested Struct Arrays | Encapsulated Banking Methods | Audit Trail Tracking | Complete Module Capstone

| Transaction History Log | Account Audit Statement |
| --- | --- |
| Init: $1,000 \| Deposit: $500 \| Withdraw: $200 | Acc #1001 (Gaurav) \| Final Balance: $1,300.00<br>Total Transactions Logged: 2 Records |